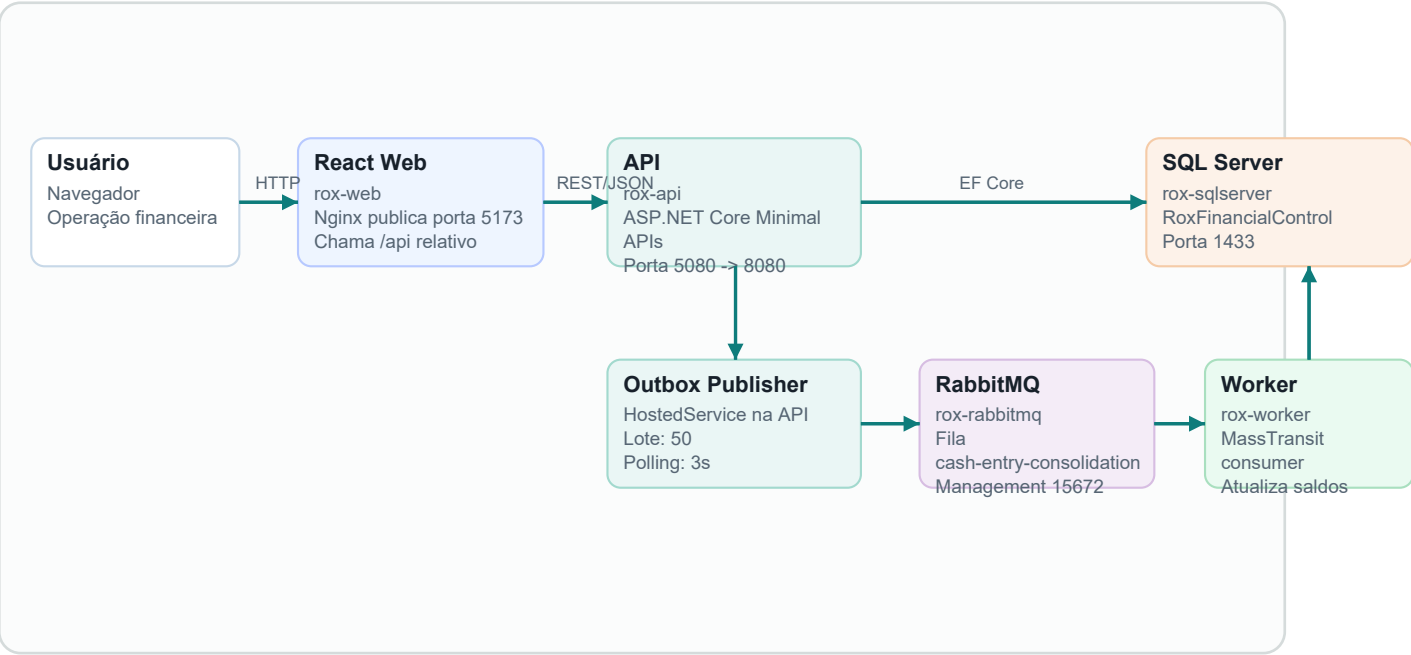


ROX Financial Control

Documento de detalhamento da arquitetura da solução, incluindo frontend, API, worker, banco de dados, RabbitMQ, containers e fluxo de consolidação assíncrona.



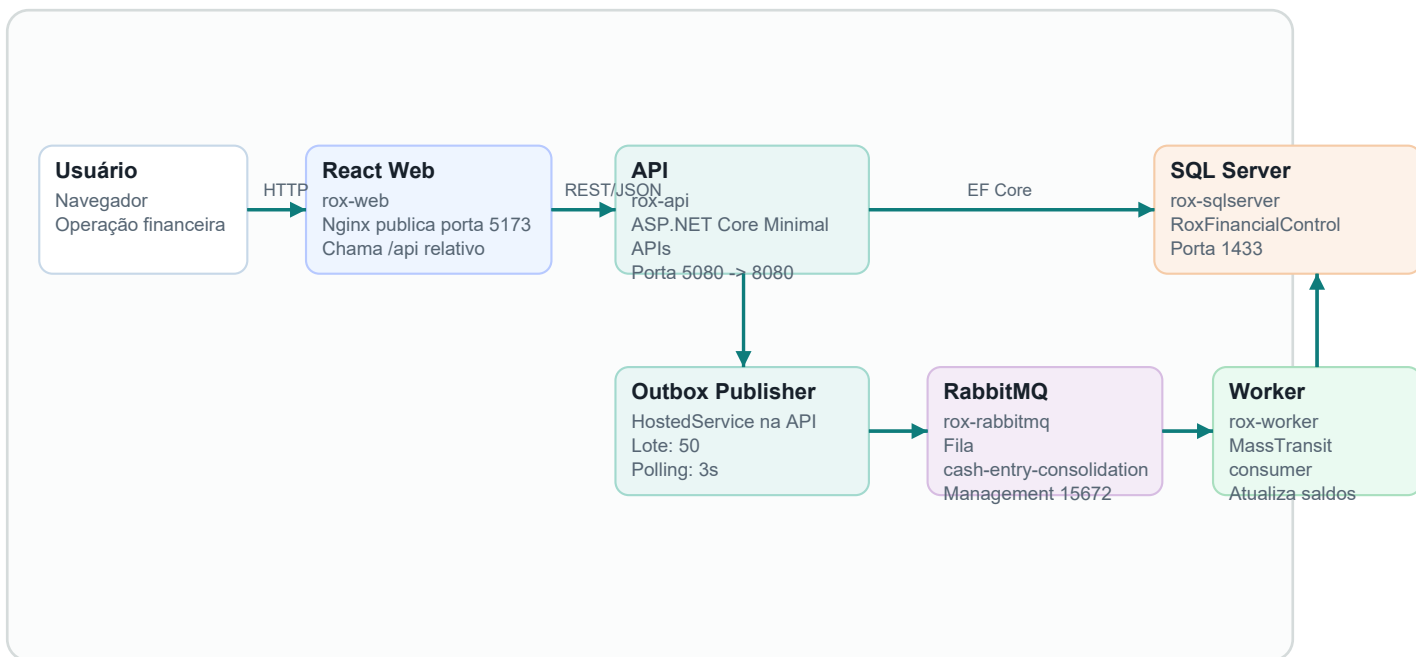
Objetivo arquitetural: permitir que a aplicação de gestão continue recebendo lançamentos mesmo quando a consolidação estiver indisponível, mantendo rastreabilidade por outbox, mensageria e consumidor idempotente.

Área	Tecnologias e responsabilidades
Frontend	React, TypeScript, Vite, TanStack Query, React Hook Form, Zod e Nginx em container.
Backend API	.NET 10, ASP.NET Core Minimal APIs, handlers de aplicação, OpenAPI, ProblemDetails e CORS.
Assíncrono	Outbox persistente na API, RabbitMQ com MassTransit e worker de consolidação.
Persistência	SQL Server com EF Core, tabelas transacionais e agregados diários.
Execução	Docker Compose com web, api, worker, sqlserver e rabbitmq.

1. Visão geral da solução

A solução foi construída como um sistema de controle de caixa com separação entre experiência web, API transacional e consolidação assíncrona. O cadastro de créditos e débitos acontece de forma síncrona na API, enquanto o cálculo do saldo diário é executado por um worker desacoplado por RabbitMQ.

Essa separação atende ao requisito de manter a aplicação de gestão operante mesmo se a consolidação falhar temporariamente. A API grava o lançamento e o evento de outbox no SQL Server; a publicação para o RabbitMQ é feita por um serviço em background com tentativas posteriores.



Principais decisões

- Separação por projetos: Domain, Application, Infrastructure, Api, Worker e Tests.
- Persistência relacional no SQL Server para lançamentos, outbox, saldos diários e idempotência.
- Mensageria RabbitMQ com MassTransit para desacoplar entrada de dados e consolidação.
- Frontend servido por Nginx e chamadas relativas para /api, evitando CORS e problemas de localhost no Docker.

2. Frontend - React Web

O projeto web fica em `src/frontend/rox-financial-control-web` e entrega a interface operacional do desafio. Ele permite filtrar período, cadastrar novo lançamento, listar lançamentos, acompanhar KPIs de saldo consolidado e consultar o status da outbox.

Elemento	Responsabilidade
App.tsx	Compõe a tela principal, filtros de período, painéis de resumo, formulário e tabela.
api/client.ts	Centraliza chamadas HTTP para /api, serialização JSON e tratamento de erro.
CashEntryForm	Valida e envia novos créditos/débitos. Após sucesso, invalida queries relacionadas.
CashEntryTable	Lista lançamentos por período, pagina e tipo quando aplicável.
BalancePanel	Consulta saldos diários consolidados e calcula a visualização de saldo.
OutboxPanel	Exibe pendências e erros recentes do mecanismo de publicação assíncrona.

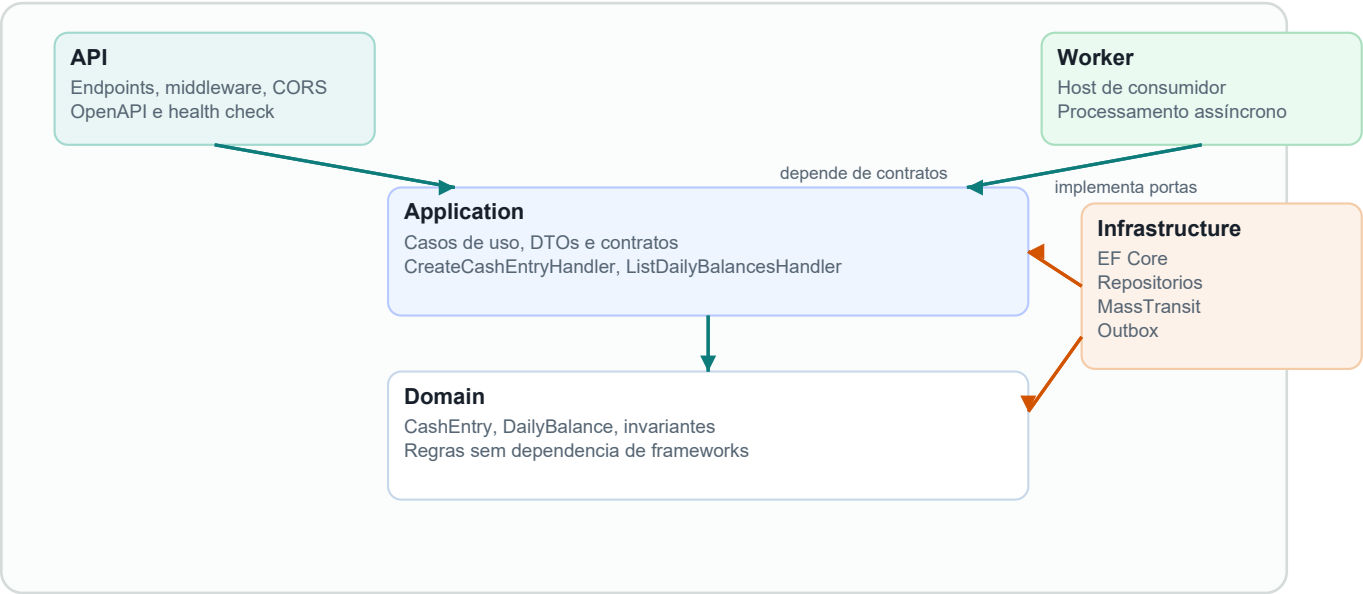
Comportamento de rede

Em Docker, o navegador acessa o Nginx no container web pela porta 5173. O frontend chama rotas relativas como `/api/daily-balances`, e o Nginx faz proxy interno para `api:8080`. Esse desenho remove a necessidade de expor detalhes da rede Docker ao browser.

Endereço recomendado em desenvolvimento: usar `http://127.0.0.1:5173` para evitar inconsistências de resolução de localhost no Windows/Docker.

3. Backend - camadas e projetos

O backend segue uma Clean Architecture pragmática. O domínio concentra regras e invariantes; Application orquestra casos de uso por meio de contratos; Infrastructure implementa persistência, mensageria e integrações; Api e Worker sao hosts de entrada.



Projeto	Papel arquitetural
Rox.FinancialControl.Domain	Entidades CashEntry e DailyBalance, tipos de domínio e exceções de regra.
Rox.FinancialControl.Application	Handlers, DTOs, queries, validações e interfaces de repositórios/outbox/clock/unit of work.
Rox.FinancialControl.Infrastructure	EF Core, SQL Server, repositórios, DbContext, outbox, MassTransit e consumidor.
Rox.FinancialControl.Api	Minimal APIs, middleware de erro, CORS, OpenAPI, health e inicialização do banco.
Rox.FinancialControl.Worker	Host .NET para consumir eventos e executar consolidação diária.
Rox.FinancialControl.Tests	Testes unitários de domínio e de caso de uso de criação de lançamento.

4. API transacional

A API é o ponto de entrada para o frontend. Ela expõe endpoints REST em `/api/cash-entries`, `/api/daily-balances` e `/api/operations/outbox`. O Program.cs registra Application, Infrastructure, OpenAPI, ProblemDetails, CORS e middleware de exceção.

Endpoint	Uso
POST <code>/api/cash-entries</code>	Cria um lançamento de crédito ou débito e registra o evento correspondente na outbox.
GET <code>/api/cash-entries</code>	Lista lançamentos com filtros de período, tipo, pagina e tamanho.
GET <code>/api/cash-entries/{id}</code>	Consulta um lançamento individual.
GET <code>/api/daily-balances</code>	Consulta saldos diários já consolidados pelo worker.
GET <code>/api/operations/outbox</code>	Mostra pendências de publicação e erros recentes da outbox.
GET <code>/health</code>	Valida disponibilidade básica da API.

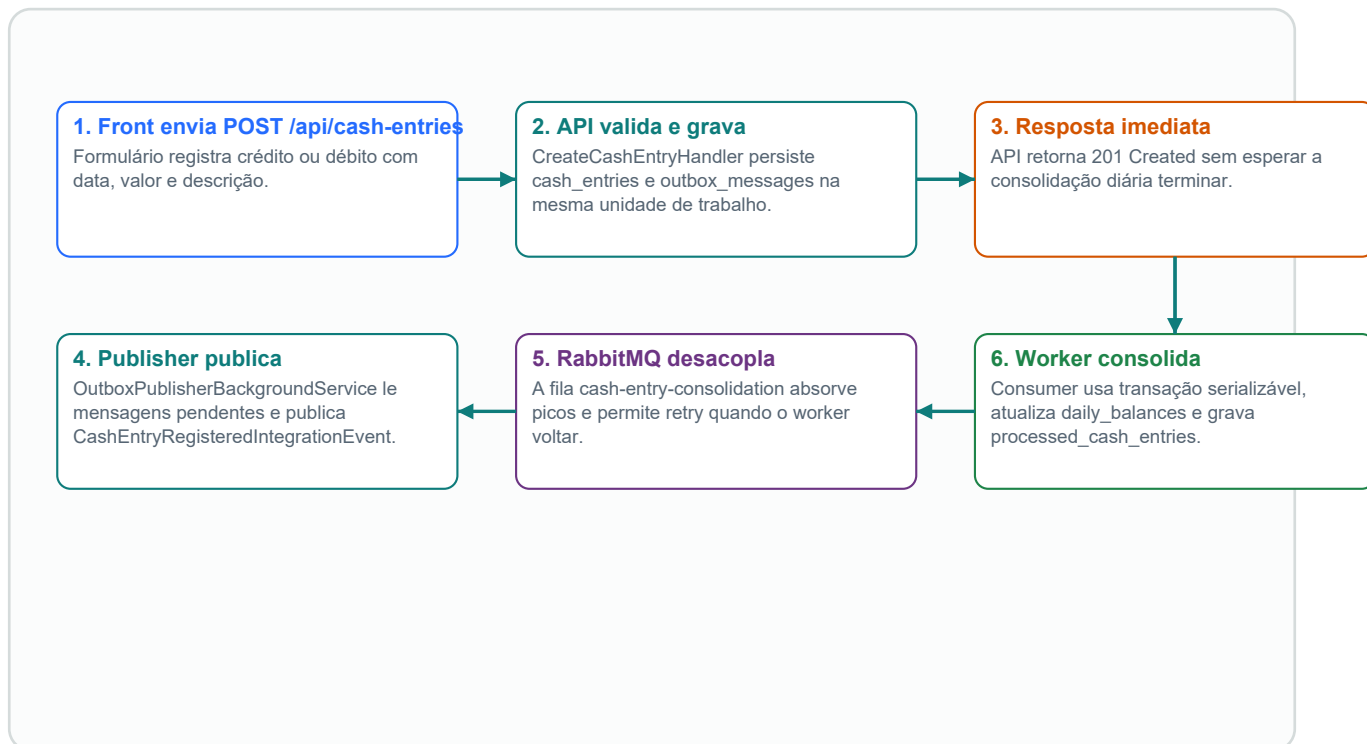
Criação de lançamento

CreateCashEntryHandler valida o tipo Credit/Debit, cria CashEntry com as datas corretas, adiciona o lançamento pelo repositório, cria um CashEntryRegisteredIntegrationEvent no outbox e salva tudo pelo UnitOfWork.

Ponto chave: a API responde 201 Created após persistir o lançamento e o registro de outbox. A consolidação diária não bloqueia a resposta do usuário.

5. Fluxo assíncrono com Outbox, RabbitMQ e Worker

O fluxo assíncrono foi escolhido para proteger a experiência de cadastro contra falhas de consolidação e para absorver picos de entrada. A API grava uma mensagem em outbox_messages e um serviço em background publica os eventos pendentes no RabbitMQ.



Componente	Detalhe implementado
OutboxPublisherBackgroundService	Executa em background na API, lê mensagens pendentes, publica via IPublishEndpoint e marca ProcessedAt.
OutboxOptions	BatchSize configurado como 50 e PollingIntervalSeconds como 3 no docker-compose.
RabbitMQ	Fila de consolidação chamada cash-entry-consolidation, exposta em 5672 e management em 15672.
MassTransit	Configura transporte RabbitMQ, publicação de eventos e endpoint consumidor no worker.
CashEntryRegisteredConsumer	Consome o evento, aplica consolidação e registra idempotência em processed_cash_entries.

Garantias operacionais

- Se RabbitMQ ou worker estiverem indisponíveis, a API continua salvando lançamentos e mantendo eventos pendentes na outbox.
- Se uma mensagem for entregue duas vezes, processed_cash_entries impede dupla aplicação no saldo diário.
- O consumidor usa retry exponencial e transação serializável para reduzir risco em concorrência.

6. Modelo de dados

O banco RoxFinancialControl no SQL Server armazena tanto dados de negócio quanto informações operacionais de mensageria. O saldo apresentado pela API é calculado a partir de $\text{TotalCredits} - \text{TotalDebits}$.

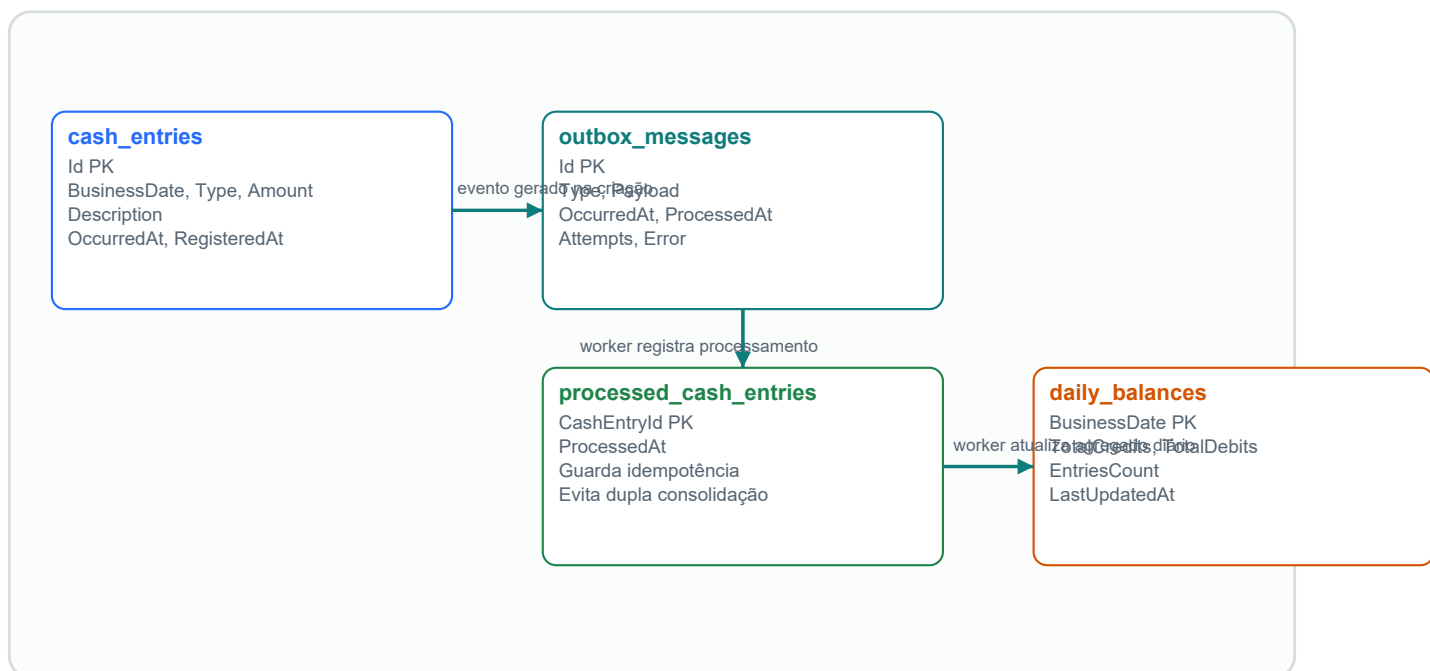


Tabela	Descrição
cash_entries	Registro imutável do lançamento financeiro original: data de negócio, tipo, valor, descrição e datas.
outbox_messages	Eventos ainda não publicados ou histórico de publicação, com tentativas e erro quando houver falha.
daily_balances	Agregado por BusinessDate com TotalCredits, TotalDebits, EntriesCount e LastUpdatedAt.
processed_cash_entries	Controle de idempotência por CashEntryId já consolidado pelo worker.

Persistência e inicialização

A Infrastructure registra `ApplicationDbContext` com `UseSqlServer` e aplica configurações via Fluent API. Para simplificar a execução local do desafio, `EnsureCreatedAsync` inicializa o schema automaticamente; em produção, a evolução natural seria substituir por migrations versionadas.

7. Containers e execução local

O docker-compose.yml sobe todos os componentes necessários para execução integrada: SQL Server, RabbitMQ, API, worker e web. API e worker usam nomes de serviço internos para acessar sqlserver e rabbitmq dentro da rede Docker.

Servico	Container	Imagem/build	Portas	Responsabilidade
web	rox-web	Build React + Nginx	5173 -> 80	Servir SPA e fazer proxy de /api para api:8080.
api	rox-api	Dockerfile.api	5080 -> 8080	Receber REST, gravar lançamentos e publicar outbox.
worker	rox-worker	Dockerfile.worker	-	Consumir fila e consolidar saldos diários.
sqlserver	rox-sqlserver	mcr.microsoft.com/mssql/server:2022-latest	1433 -> 1433	Banco RoxFinancialControl e volumes persistentes.
rabbitmq	rox-rabbitmq	rabbitmq:3.13-management	5672, 15672	Broker AMQP e painel de gerenciamento.

Comandos uteis

Acao	Comando
Subir tudo	docker compose up -d --build
Ver status	docker compose ps
Logs da API	docker compose logs api --tail 120
Listar filas	docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
Consultar banco	SSMS em 127.0.0.1,1433 com usuário sa.

8. Resiliência, escalabilidade e observabilidade

A arquitetura prioriza isolamento de falhas. O fluxo de entrada permanece disponível mesmo se a consolidação estiver atrasada; a fila absorve variações de carga e o worker processa em paralelo controlado.

Cenário	Comportamento esperado
Worker parado	Lançamentos continuam sendo gravados; mensagens ficam na fila ou outbox até o worker voltar.
RabbitMQ indisponível	Outbox permanece pendente; o publisher tenta novamente a cada ciclo.
Mensagem duplicada	processed_cash_entries identifica CashEntryId já consolidado e evita dupla contagem.
Picos de requisições	API grava rapidamente no SQL; consolidação segue pelo ritmo da fila e do worker.
Falhas transitorias no SQL/Rabbit	MassTransit e execution strategy do EF Core ajudam nos retries configurados.

Pontos de monitoramento

- GET /api/operations/outbox para total de pendências e últimos erros de publicação.
- RabbitMQ Management em 15672 para Publish, Deliver, Ack, Ready, Unacked e consumidores.
- Logs de api e worker para erros de persistência, publicação ou consumo.
- SQL Server para validar cash_entries, daily_balances, outbox_messages e processed_cash_entries.

9. Testes e evolução

A base inclui testes unitários de domínio e aplicação. Eles cobrem invariantes de CashEntry, regras de DailyBalance e o caso de uso que cria lançamento e registra evento de outbox.

Área	Cobertura atual ou recomendada
Domain	Valida montante positivo, tipos válidos e acumulação de crédito/débito em DailyBalance.
Application	Garante que CreateCashEntryHandler persiste o lançamento e adiciona evento de outbox.
Integration tests	Recomendado adicionar Testcontainers para SQL Server e RabbitMQ.
Load tests	Recomendado validar 50 req/s, latência da API e atraso máximo da consolidação.
Observabilidade	Proximo passo: OpenTelemetry, métricas de fila e dashboards de erro da outbox.

Resumo

A solução adotada entrega uma separação clara de responsabilidades: frontend para operação, API para transação, SQL Server para verdade persistida, RabbitMQ para desacoplamento e worker para consolidação. O padrão outbox e a idempotência do consumidor reduzem perda de dados e tornam o fluxo tolerante a falhas.